

Cahier des Charges

Système de Réservation de Billets d'Avion

Client : Compagnie aérienne (projet pédagogique — ECUE IIAA511, 2iE)

Prestataire : Équipe de développement — Lamine ZARE, Fatoumata DIABATE, Tariq TRAORE

Version 1.0

Table des matières

1	Exigences du client	3
2	Exigences techniques imposées par le client	3
3	Architecture proposée par le prestataire	4
4	Règles de gestion contractuelles	5
5	Recette — scénarios de validation exigés par le client	5
6	Organisation de l'équipe prestataire	5
7	Livrables contractuels	6
8	Critères d'acceptation par le client	6

1. Exigences du client

Le client formule les exigences suivantes, que le prestataire s'engage à satisfaire intégralement :

- Le système doit afficher les informations d'un vol (numéro, aéroports de départ et d'arrivée, prix, places disponibles) en un seul appel.
- Le système ne doit jamais permettre de survendre un vol : toute tentative de réservation au-delà des places disponibles doit être refusée sans faire planter le programme.
- Le système doit appliquer automatiquement un avantage tarifaire à chaque passager selon son statut de fidélité, sans intervention manuelle.
- Chaque réservation doit relier de façon unique un numéro de billet, un vol et un passager identifiés.
- Le client doit pouvoir confirmer une réservation et en consulter l'état à tout moment.
- La logique de réservation et de libération ne doit pas être dupliquée entre la gestion des vols et celle des réservations ; elle doit être factorisée et réutilisable.
- Toute opération refusée (place épuisée, double réservation, donnée invalide) doit laisser le système dans un état inchangé et signaler l'échec de façon exploitable, sans interruption brutale du programme.
- L'architecture livrée doit rester ouverte à l'ajout futur de nouvelles catégories de passagers ou de nouveaux types de ressources réservables, sans modification du code existant.

2. Exigences techniques imposées par le client

Le client, sensibilisé aux bonnes pratiques de conception orientée objet, impose les contraintes suivantes à l'équipe de développement :

Exigence technique	Raison invoquée par le client
La notion de « passager » doit être représentée par une classe abstraite , non instanciable directement.	Le client veut garantir qu'un passager appartient toujours à une catégorie précise et jamais à une catégorie générique indéfinie.
Chaque catégorie de passager doit redéfinir son propre calcul d'avantage/réduction.	Permet au client d'ajouter facilement une nouvelle catégorie plus tard sans toucher au code existant.
La capacité « réserver / annuler / calculer un prix » doit être exprimée par une interface , implémentée par la réservation, qui s'appuie en interne sur le vol et le passager qu'elle contient.	Le client ne veut pas que cette logique soit dispersée : la réservation reste le point d'entrée unique de la capacité, tout en déléguant aux objets qu'elle contient.
Toute donnée invalide transmise à la création d'un objet doit être rejetée immédiatement (le système ne doit jamais accepter un objet dans un état incohérent).	Exigence de fiabilité du client : « pas de code fidélité vide, pas de vol avec des places négatives ».

Exigence technique	Raison invoquée par le client
Le prototype doit démontrer, par du code exécutable, que le système traite les objets de façon générique (sans code spécifique à chaque catégorie).	Le client veut la preuve que l'architecture est réellement extensible, pas seulement sur le papier.

3. Architecture proposée par le prestataire

```

<<interface>>
Reservable
+ reserver() : boolean
+ annuler() : boolean
+ calculerPrix() : double
    ^
    | implemente
    |
    Reservation
- numero_billet : int
- vol : Vol
- passager : Passager
- confirm_vol : boolean
+ Reservation(int, Vol, Passager, boolean)
+ reserver() : boolean
+ annuler() : boolean
+ calculerPrix() : double
+ confirm_reservation() : void
+ afficher_reservation() : void
    ^               ^
    | contient      | contient
    |               |
Vol                 Passager (abstraite)
- numero_vol : String    # nom, prenom, passeport : String
- depart, arrive         + Passager(String, String, String)
- prix : int              + getNom() / getPrenom() / getPasseport()
- place_dispo : int       + afficherPassager() : void
+ reserver_place() : bool  + calculerReduction() : double (abstraite)
+ annuler_place() : void
+ getPrix() / getPlaceDispo()
+ afficher() : void

                                     ^
                                     | herite
                                     +-----+-----+
                                     |               |
                                PassagerStandard   PassagerVIP
                                - siegeAssigne      - codeFidelite
                                + calculerReduction()->0.0  + calculerReduction()->0.15

```

Contrairement à une première ébauche envisagée, l'interface **Reservable** n'est implémentée que par **Reservation** : c'est elle qui porte la capacité complète (réserver, annuler, calculer le prix), en s'appuyant en interne sur le **Vol** et le **Passager** qu'elle contient par compo-

sition (losanges pleins sur le diagramme). `Vol` garde sa propre logique de gestion des places (`reserver_place()`, `annuler_place()`), mais n'implémente pas l'interface lui-même.

4. Règles de gestion contractuelles

Le prestataire s'engage à faire respecter les règles suivantes par le système livré :

- Le nombre de places disponibles d'un vol ne peut jamais devenir négatif.
- Un passager grand voyageur (VIP) ne peut pas exister sans code de fidélité valide.
- Une réservation confirmée ne peut pas être réservée une seconde fois sans annulation intermédiaire.
- Une réservation déjà annulée ne peut pas être annulée une seconde fois (l'opération est refusée, pas ignorée silencieusement).
- Le calcul du prix d'une réservation (`calculerPrix()`) doit toujours refléter le prix du vol et l'avantage tarifaire réel du passager qu'elle contient.

5. Recette — scénarios de validation exigés par le client

Avant livraison, le prestataire doit démontrer, via un programme de démonstration, les scénarios suivants :

Type de scénario	Cas à démontrer
Cas nominal	Réservation d'une place sur un vol disponible ; confirmation d'une réservation valide
Cas limite	Réservation de la toute dernière place disponible d'un vol
Cas d'échec	Réservation d'un vol complet ; création d'un passager VIP sans code de fidélité ; annulation d'une réservation déjà annulée
Preuve d'extensibilité (polymorphisme)	Traitement d'une liste mêlant plusieurs catégories de passagers sans code spécifique à chacune ; manipulation de plusieurs réservations via la seule interface Reservable , sans connaître leur vol ou leur passager réel

6. Organisation de l'équipe prestataire

Membre	Lot confié
Lamine ZARE	Développement des classes Vol et Reservation , compilation du livrable
Fatoumata DIABATE	Développement de la classe Passager et de ses catégories, documentation (README.md)
Tariq TRAORE	Recette : scénarios de test et démonstration dans le programme principal

7. Livrables contractuels

- Code source Java complet, compilable et exécutable.
- Diagramme de classes à jour (héritage en trait plein, implémentation en trait pointillé, classe abstraite clairement identifiée).
- Documentation (`README.md`) justifiant les choix d'architecture retenus et un choix écarté.
- Programme de démonstration exécutant au minimum quatre scénarios de recette (deux nominaux, deux limites/échec).
- Historique de versions (dépôt Git) traçant la contribution de chaque membre de l'équipe.

8. Critères d'acceptation par le client

- ☐ Le programme compile et s'exécute sans erreur.
- ☐ Aucune instance directe de la classe abstraite `Passager` n'existe dans le code.
- ☐ Le traitement générique (polymorphisme) est visible et démontrable dans le programme de démonstration.
- ☐ Toute donnée invalide est rejetée à la création des objets concernés.
- ☐ Le diagramme livré correspond exactement au code livré.
- ☐ La documentation justifie chaque choix de conception structurant.